

Quicksort: Splitting an Unsorted Pile of Books into Sorted Piles

Remember that `binarySearch()`'s speed advantage comes from the fact that the values in items are sorted. If the values are out of order, the algorithm won't work. Enter *quicksort*, a recursive sorting algorithm developed by computer scientist Tony Hoare in 1959.

Quicksort uses a divide-and-conquer technique called *partitioning*. Think of partitioning this way: imagine you have a large pile of unalphabetized books. Grabbing one book and placing it in the right spot on the shelf means you'll spend a lot of time rearranging the bookshelf as it gets full. It would help if you first turned the pile of books into two piles: an *A* to *M* pile and an *N* to *Z* pile. (In this example, *M* would be our *pivot*.)

You haven't sorted the pile, but you have *partitioned* it. And partitioning is easy: the book doesn't have to go into the correct place in one of the two piles, it just has to go into the correct pile. Then you can further partition these two piles into four piles: *A* to *G*, *H* to *M*, *N* to *T*, and *U* to *Z*. This is shown in [Figure 5-2](#). If you keep partitioning, you end up with piles that contain one book each (the base case), and the piles are now in sorted order. This means the books are now in sorted order as well. This repeated partitioning is how quicksort works.

For the first partitioning of *A* to *Z*, we select *M* as the pivot value because it's the middle letter between *A* and *Z*. However, if our collection of books consisted of one book about Aaron Burr and 99 books about zebras, zephyrs, zoos, zygotes, and other *Z* topics, our two partitioned piles would be heavily unbalanced. We would have the single Aaron Burr book in the *A* to *M* pile and every other book in the *M* to *Z* pile. The quicksort algorithm works fastest when the partitions are evenly balanced, so selecting a good pivot value at each partition step is important.